

63. Long Polling

Server ke saath **persistent connection** rakhne ka sabse simple tarika, jisme WebSocket ya Server Sent Events jaisa koi special protocol nahi lagta. Implement karna aasan hai aur kaafi cases me kaafi hota hai.

Regular Polling

Har kuch second (jaise 10 sec) me server se puchhna: "Kuch naya hai?"

Kaam karta hai, par kamiyan:

1. Messages me **10 second tak ki der** ho sakti hai.
2. Message na ho tab bhi server par har 10 second me requests aati hain, chahe user kahin aur chala gaya ho ya soya ho. Performance ke hisaab se load hai.

Chhoti service ke liye theek, par aam taur par behtar tarika chahiye.

Long Polling

Ye kahin behtar hai, aasan hai, aur **bina delay** ke messages deta hai.

Flow:

1. Server ko request bhejo.
2. Server connection **tab tak band nahi karta** jab tak bhejne ko message na ho.
3. Message aate hi server request ka jawab de deta hai.
4. Browser **turant nayi request** bhejta hai.

Yaani browser request bhej ke server ke saath pending connection rakhta hai (yahi is method me normal hai). Sirf message milne par connection band hokar dobara banta hai. Network error se connection toote to bhi browser turant nayi request bhejta hai.

js

```

async function subscribe() {
  let response = await fetch("/subscribe");

  if (response.status == 502) {
    // connection timeout (bahut der pending raha, server ya proxy ne band kiya)
    await subscribe(); // reconnect
  } else if (response.status != 200) {
    showMessage(response.statusText); // error dikhao
    await new Promise(resolve => setTimeout(resolve, 1000)); // 1 sec baad
    await subscribe();
  } else {
    let message = await response.text();
    showMessage(message);
    await subscribe(); // agla message
  }
}

subscribe();

```

`subscribe` fetch karta hai, response ka wait karta hai, handle karta hai, aur khud ko phir call karta hai.

Server ko bahut saare pending connections sambhalne chahiye

Kuch server architectures har connection ke liye ek alag **process** chalte hain (jaise PHP, Ruby ke kai setups), aur har process kaafi memory leta hai. Bahut connections hone par memory khatam ho sakti hai. **Node.js** jaise servers me aam taur par dikkat nahi. Ye language ki problem nahi, architecture ki hai. Bas dhyan rakho ki server bahut saare simultaneous connections theek se handle kare.

Demo: Chat

- **Message bhejna:** simple `POST`.

```

js
fetch(url, { method: 'POST', body: message });

```

- **Message lena:** upar wala `subscribe` (long polling).

Server ka idea (Node.js):

- Har subscriber ka response object `subscribers[id] = res` me rakho, band mat karo.
- Client disconnect ho (`req.on('close')`) to hata do.

- `publish(message)` par sabhi `subscribers` ko `res.end(message)` karo aur list khaali kar do.

Kab use karein?

Long polling **tab achha hai jab messages kabhi kabhi aate hon.**

Agar messages bahut jaldi jaldi aate hon, to request-response ka graph "aari" (saw) jaisa ho jaata hai: har message ek alag request hai, headers aur authentication ka overhead ke saath. Aise cases me **WebSocket** ya **Server Sent Events** behtar hain.